

Modulo 1

Fundamentos de programación y Lenguajes

Una computadora sola no hace nada, es básicamente un cascarrón. Necesitas programas (listas de instrucciones bien simples, tipo "divide este número y guarda el resultado") para poder hacer cosas más complejas.

El problema es que nosotros no hablamos lenguaje máquina. Por eso existen los lenguajes de alto nivel (el código fuente), que funcionan como intermediario entre lo que escribimos y lo que la máquina realmente entiende.

Todo lenguaje, sea humano o de máquina tiene, entre otras partes: un alfabeto (los símbolos) o léxico (las palabras que existen), una sintaxis (cómo se acomodan esas palabras) y una semántica (que significa cuando ya están acomodadas).

Compilación y Interpretación

Hay dos formas de pasar de código de alto nivel a lenguaje máquina.

Compilación → Traduce todo el código de una sola vez y te deja un ejecutable, corre rápido y nadie ve tu código fuente, pero tienes que compilar distinto para cada plataforma.

Interpretación → El intérprete va leyendo línea por línea y ejecutando sobre la marcha. Si hay un error se detiene ahí mismo y te avisa.

Python

Lo creó Guido van Rossum en 1989, según el mismo cuenta como proyecto para coltarminicatos durante las vacaciones. El nombre no tiene que ver con una serpiente, es por el grupo de comedia Monty Python.

Es gratis, de código abierto y corre en cualquier sistema.
Desde el principio la idea era que fuera fácil de aprender, fácil de escribir y que se leyera casi como inglés normal.

Se usa muchísimo en desarrollo web, ciencia de datos, pruebas de software y scripts del día a día. Normalmente se le compara con Perl y Ruby. Eso sí, no es el lenguaje más rápido que existe y casi nadie lo usa para programación de bajo nivel o para apps móviles nativas.

Versiones

CPython: la oficial, escrita en C, mantenida por la Python Software Foundation. Es la que casi todas usamos.

Cython: Traduce Python a C para que los cálculos matemáticos pesados corran mucho más rápido.

Jython: Versión en Java, útil si necesitas meter Python dentro de un sistema grande hecho en Java.

Modulo 2

Python se ve facil por fuera, pero por dentro tiene reglas bien definidas sobre como manejar los datos, las variables y los operadores. Aqui va un repaso de lo basico

Literales

Un literal es simplemente es un dato escrito en el codigo. Se escribe solo, sin necesidad de contexto de nada mas

Python maneja varios

Enteros (int) numeros sin parte decimal positivos, negativos o cero. No se puede usar comas ni puntos para separar miles, pero si guiones bajos si el numero es grande. 1.000.000 se lee como un millon normal

Flotantes (float) numeros con decimales siempre con punto, nunca con coma. Se puede omitir la parte entera o la decimal si es cero. Para numeros muy grandes o muy chicos se usa notacion cientifico con E o e, aunque el exponente despues de la E tiene que ser un entero si o si

Cadenas (strings) texto entre comillas simples o dobles. Esto sirve para evitar comillas sin que se rompa el codigo, por ejemplo el libro "Don quixote" se puede escribir con comillas simples por fuera y dobles dentro

Booleanos y None. True y False son los valores booleanos y hay que escribirlos exactamente asi con mayuscula inicial o marca error. None se usa para decir que ohi no hay ningun valor

2-Variables

Si los literales son la materia prima, las variables son donde los guardas. En Python no hay que declarar variables antes, se crea sola en el momento en que le asignas algo con el signo =
Y como son dinámicas puedes cambiarle cualquier tipo de dato y cambiarlo después sin problemas.

Para nombrar variables hay reglas que hay que seguir SI O SI°

- Solo letras, números y guion bajo
- No puede empezar con número
- Nada de espacios
- var y Var son variables distintas
- No se pueden usar palabras reservadas

3-Operadores y jerarquía

Una expresión es cualquier combinación de valores, variables y operadores que al evaluarse da un resultado. Cuando hay varios operadores juntos Python decide el orden según su prioridad

1- Unarios (+, - antes de un solo número como -1)

2- Exponenciación

3- Multiplicación, división

4- Suma o resta

Cuando dos operadores tienen la misma prioridad se resuelve de izquierda a derecha

4-Interacción con el usuario y comentarios

Un programa que no recibe nada del usuario es un programa sordo. Para eso está `input()`: para el programa, espera al que el usuario escribe algo y devuelve antes y devuelve ese valor. A diferencia de `print` que solo muestra cosas en pantalla y no devuelve nada importante. Ese valor hay que guardarlo en una variable, si no se pierde.

`input()` puede recibir mensaje como argumento, por ejemplo `input("Dime algo")` muestra ese texto antes de esperar la respuesta. Pero eso lo que regresa `input` siempre es en string, sin importar que haya escrito el usuario. Si lo intentas usar directo en una operación matemática, el programa tiene error, primero hay que convertirlo a `int` o `float`.

Por último los comentarios. Empiezan con `#` y van hasta el final de esa línea; si necesitas varios líneas cada una lleva su propia `#`. Sirven para explicar código complicado, de ser constante, de quien hizo que y cuando, o simplemente para acordarte lo mismo de que hace tu código después de un tiempo sin tocarlo.

Modulo 3

1-Bases de la logica computacional

Los computadores evalúan el flujo de un programa a partir de "Preguntas" que solo admiten dos respuestas booleanas: Verdadero (True) falso (False)

2-Operadores en Python

A-Operadores de comparacion

Comparan dos operandos y devuelven un valor booleano. Se evalúan de izquierda a derecha

Operador	Descripcion
<code>==</code>	Igual a
<code>!=</code>	Diferente de
<code>></code>	Mayor que
<code>>=</code>	Mayor o igual que
<code><</code>	Menor que
<code><=</code>	Menor o igual que

B-Operadores logicos

Sirven para construir condiciones complejas combinando varias expresiones

- `and` (conjuncion)
- `or` (disyuncion)
- `not` (negacion)

C-Operadores bit a bit

Operan directamente sobre la representación binaria de los números enteros

- `&` (AND)
- `|` (OR)
- `^` (XOR)
- `~` (Not)
- `>>` / `<<` desplazamiento de bits

3- Control de flujo:

Ejecutan un bloque de código solo cuando se cumple una condición. Python exige sangría (indentación) consistente para delimitar cada bloque.

- if: Se evalúa de forma independiente
- else: va emparejado con un if
- elif: Permite encadenar varias condiciones

4- Bucles

Repeten un bloque de código las veces que nos haga falta

A- Bucle while

Corre mientras la condición sea True. Hay que modificar esa condición dentro del bucle, o se vuelve infinito. Un bucle infinito se puede detener manualmente

B- Bucle for

Pensado para iterar sobre una secuencia (listas, cadenas, rangos, etc.)

C- Alteración del flujo de un bucle

- break: Sale de bucle de inmediato
- Continuar: Salta el resto de la iteración actual y pasa a la siguiente

5- Estructuras de Datos: listas

Guardan varios valores en una sola variable

- Sintaxis: Se delimita con corchetes `[...]`, elementos separados por comas
- Indexación: empieza en 0 = el primer elemento tiene índice
- Tipo dinámico: una misma lista puede mezclar tipos (enteros, flotantes, otras listas, etc.)

6- Algoritmos y pseudocódigo

- Pseudocódigo: forma de escribir un algoritmo sin atarse a la sintaxis de un lenguaje específico.
- Funciones integradas útiles: `Max()` devuelve el valor más grande y `min()` el más pequeño

Funciones en Python

Sirven para simplificar tareas repetitivas y organizar mejor el código

- Evitar la repetición del código: Si un bloque aparece más de una vez, conviene aislarlo en una función. Así si hay que corregir algo se corrige en un solo lugar

- Descomposición: un problema grande se divide en piezas aisladas y cada una se codifica y se prueba por separado

- Trabajo en equipo: Permite repartir un proyecto grande entre varios desarrolladores, y hace el código más legible sin depender de tantas comentarios

2-¿ De donde provienen las funciones

En Python hay tres funciones principales

- Integradas (built-in) forman parte del lenguaje y siempre están disponibles

- De modelos preinstalados: Se obtiene el `import` modelos que vienen con Python

- Definidas por el usuario: las escribe el programador con la palabra clave `def`.

Funcionamiento y reglas de ejecución

- Invocación: al llamar a función, Python recuerda el punto actual, salta el cuerpo de la función, ejecuta sus instrucciones y regresa justo después de la llamada